TranCIM: Full-Digital Bitline-Transpose CIM-based Sparse Transformer Accelerator With Pipeline/Parallel Reconfigurable Modes

Fengbin Tu, *Member, IEEE*, Zihan Wu, Yiqi Wang, Ling Liang, Liu Liu, *Member, IEEE*, Yufei Ding, Leibo Liu, *Senior Member, IEEE*, Shaojun Wei, *Fellow, IEEE*, Yuan Xie, *Fellow, IEEE*, and Shouyi Yin, *Member, IEEE*

***Abstract*— Transformer models achieve excellent results in the fields like natural language processing, computer vision, and bioinformatics. Their large numbers of matrix multiplications (MMs) lead to substantial data movement and computation. Although computing-in-memory (CIM) has proven to be an efficient architecture for MM computation, transformer's attention mechanism raises new challenges in memory access and computation aspects: the dynamic MM in attention layers causes redundant OFF-chip memory access; Attention layers dominate transformer's computation and require high precision. Thus, we design a bitline-transpose CIM-based transformer accelerator TranCIM with pipeline/parallel reconfigurable modes. The pipeline mode alleviates off-chip access for attention layers. The parallel mode is used by fully-connected (FC) layers for high parallelism. The full-digital CIM supports INT16 for attention layers and INT8 for FC layers, without analog CIM's nonideal issues. Moreover, a sparse attention scheduler (SAS) is proposed to reduce attention computation. The fabricated TranCIM chip only consumes 15.59 μJ/Token for the bidirectional encoder representations from transformer (BERT)-base model, achieving 12.08×–36.82× lower energy than prior CIM-based accelerators.**

***Index Terms*— Computing-in-memory (CIM), pipeline, reconfigurable architecture, sparsity, transformer.**

I. Introduction

TRANSFORMER models have achieved state-of-the-art results in many fields such as natural language processing, computer vision, and bioinformatics. In transformer models, there are large numbers of matrix multiplications (MMs) that result in substantial data movement and computation. The high latency and energy make it quite essential to design specific accelerators for transformer models.

In recent years, computing-in-memory (CIM) proves to be an efficient architecture for MM computation. By integrating multiply-accumulation (MAC) into memory such as static random access memory (SRAM) [6], [15], [23] and resistive RAM (ReRAM) [9], [20], [21], CIM eliminates the memory access bottleneck and has been applied to previous neural network (NN) accelerators.

A typical transformer model, as illustrated in Fig. 1, is usually composed of stacking multiple attention layers and fully-connected (FC) layers. The attention layers represent the attention mechanism introduced by the transformer, which are the key building blocks that contribute to state-of-the-art accuracy. However, the transformer's attention mechanism raises new challenges for CIM-based accelerator design in both memory access and computation aspects (see Fig. 2).

Challenge1a (Memory Access): Attention layers introduce dynamic MM (QK^T and $A'V$) whose weights and inputs are both generated during runtime, leading to redundant OFF-chip memory access for intermediate data. Conventional NN layers and transformer's FC layers are static MM whose weights are pre-trained and do not change during inference. Previous CIM-based accelerators usually compute MM one by one. Weights are stored in CIM only once and fully reused for each MM. However, both inputs and weights of dynamic MM are dynamically generated. In the traditional manner, the intermediate data have to be written out and then loaded back to CIM, producing redundant OFF-chip memory access.

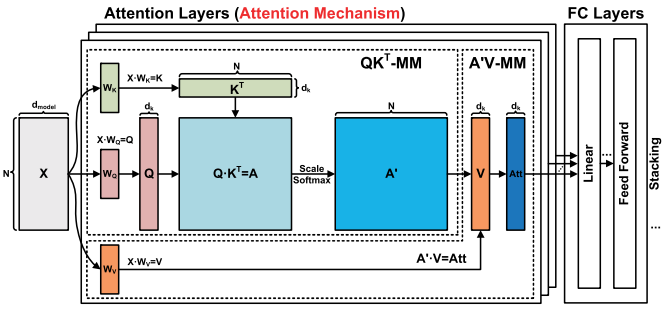


Fig. 1. Typical transformer model with stacked attention and FC layers.

Manuscript received 27 June 2022; revised 13 September 2022; accepted 5 October 2022. Date of publication 28 October 2022; date of current version 26 May 2023. This article was approved by Associate Editor Vivek De. This work was supported in part by NSFC under Grant 61774094 and Grant U19B2041, in part by the National Key Research and Development Program under Grant 2018YFB2202600, in part by the Beijing S&T Project under Grant Z191100007519016, and in part by the Beijing Innovation Center for Future Chip. *(Corresponding author: Shouyi Yin.)*

Fengbin Tu, Zihan Wu, Yiqi Wang, Leibo Liu, Shaojun Wei, and Shouyi Yin are with the School of Integrated Circuits, the Beijing Innovation Center for Future Chip, and the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: yinsy@tsinghua.edu.cn).

Ling Liang, Liu Liu, and Yuan Xie are with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA 93106 USA.

Yufei Ding is with the Department of Computer Science, University of California at Santa Barbara, Santa Barbara, CA 93106 USA.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/JSSC.2022.3213542.

Digital Object Identifier 10.1109/JSSC.2022.3213542

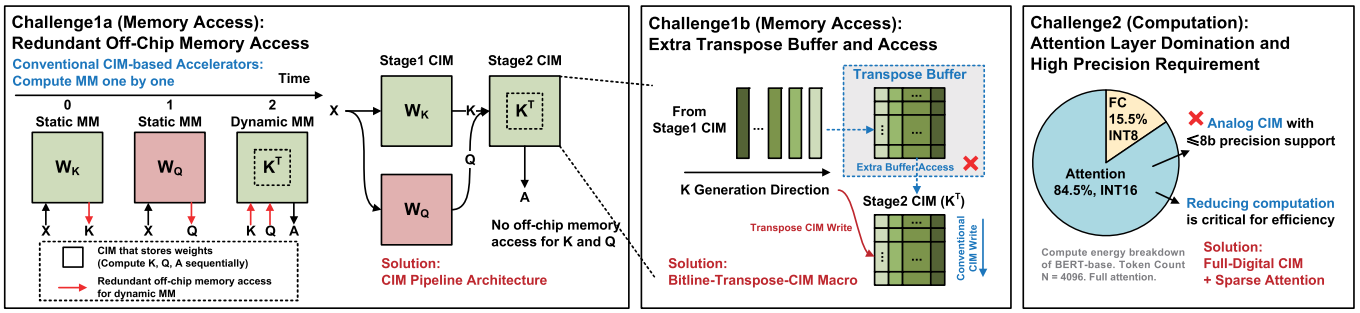


Fig. 2. Transformer's attention mechanism raises challenges for CIM-based accelerator design. TranCIM has corresponding features that target the challenges.

Challenge1b (Memory Access): Connecting CIM with a pipeline architecture can mitigate Challenge1a, but also produces a new challenge when processing the QK^T computation. Since the K generation direction does not match the conventional CIM write direction, the QK^T-pipeline needs a large transpose buffer with extra overhead.

Challenge2 (Computation): Compared with FC layers, attention layers dominate the transformer's computation and require >8b precision to maintain accuracy. However, previous analog CIM can only support ≤8b precision due to nonideal issues such as signal margin and process variation [9], [15], [20], [21], [23]. Meanwhile, reducing the computation amount of attention layers becomes more critical for efficiency improvement.

In this article, we design a full-digital CIM-based sparse transformer accelerator called TranCIM, with pipeline/parallel reconfigurable modes. TranCIM overcomes the above challenges with the following three features.

1) For **Challenge1a**, TranCIM connects its CIM engines through a reconfigurable streaming network (RSN) with dedicated modes for different layers in transformer models. For attention layers, TranCIM is configured as the pipeline mode that directly streams the 1st-stage engine's outputs to the 2nd-stage engine. For FC layers, TranCIM is configured as the parallel mode with all engines working independently.
2) For **Challenge1b**, we design the CIM macro with a bitline-transpose structure to align the directions of input feeding and weight writing, instead of the previous wordline-feeding solutions [6], [15], [23]. Thus in the QK^T-pipeline mode, transposing K is realized without additional storage and buffer access.
3) For **Challenge2**, TranCIM supports INT16 precision for attention layers and INT8 precision for FC layers through a full-digital CIM design, which totally avoids the nonideal issues of analog CIM. Meanwhile, based on recent sparse transformer algorithms [2], [25], attention layers can be trained with block sparse patterns to reduce computation while maintaining high accuracy. We design a sparse attention scheduler (SAS) to dynamically configure CIM workload for different sparse attention patterns, realizing computation complexity reduction from $O(N^2)$ to $O(N)$, where N equals the transformer's token count.

This article is the journal extension version of our International Solid-State Circuits Conference (ISSCC)'22 work [19], providing higher-level insights for readers with many additional materials. The rest of this article is organized as follows: Section II provides background and motivation. Section III proposes TranCIM's overall architecture. The three architecture features are described in Sections IV–VI, respectively. Section VII presents our experimental results on the 28 nm TranCIM chip. Section VIII concludes this article.

II. Background and Motivation

A. Transformer Model

As shown in Fig. 1, attention layers represent the attention mechanism that makes transformer models different from previous NNs and helps achieve state-of-the-art accuracy. The input of an attention layer is a sequence of N tokens (X), and each token is a vector with a length of d_{model}. By multiplying X with the corresponding weight matrices W_Q, W_K, W_V, we obtain the query (Q), key (K), and value (V) matrices with $N \times d_k$ elements. Matrix Q and the transpose of Matrix K are then multiplied to compute the attention matrix $A = Q \cdot K^T$, which represents each input token's importance to the outputs. Matrix A is normalized with a softmax function to generate Matrix $A' = \text{softmax}((A/\sqrt{d_k}))$, which is finally multiplied by Matrix V to compute the attention layer's output matrix $\text{Att} = A' \cdot V$.

However, as mentioned in **Challenge2**, attention layers dominate the transformer's computation due to the complexity of $O(N^2)$. To reduce the quadratic complexity, many sparse transformers have been proposed, in which each token can attend to only a subset of the entire tokens [2], [3], [7], [14], [25]. Fig. 3 demonstrates three typical sparse attention patterns. Jouppi et al. [7] perform sparse factorization on the attention matrix and reduce its complexity to $O(N\sqrt{N})$ with the use of two attention patterns. The strided pattern [Fig. 3(a)] attends to every l_{th} location, where l_{th} is the stride step. The fixed pattern [Fig. 3(b)] attends to specific positions defined by the algorithm. Devlin et al. use a combination of global and local attention patterns [Fig. 3(c)] to reduce the complexity to $O(N)$ [2]. In order to make good use of hardware like GPU and TPU, sparse attention can be blocked with a parameter b, which means every b token of the input sequence is processed together to compute a $b \times b$ block of Matrix A. Sparse transformers like Big Bird [25] and extended

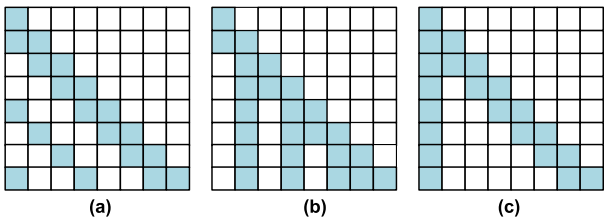


Fig. 3. Typical sparse attention patterns (dense-blue, sparse-white). (a) Strided. (b) Fixed. (c) Global + Local.

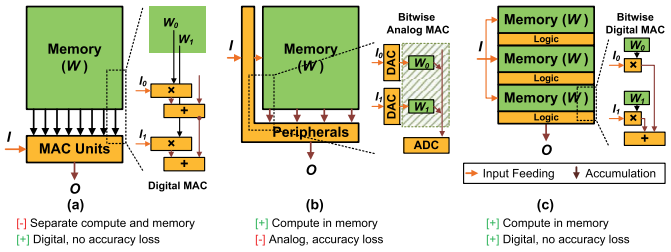


Fig. 4. Overview of different NN accelerator architectures. (a) Digital Architecture. (b) Analog CIM. (c) Digital CIM.

transformer construction (ETC) [2] have proved that block sparse attention has negligible accuracy loss while offering CIM-friendly computation patterns. The block sparsity has a less significant influence on accuracy than quantization. Thus, we will apply this technique to TranCIM for computation reduction in attention layers.

B. NN Accelerator and Computing-In-Memory

NN accelerators have witnessed significant development in recent years. Fig. 4 provides an overview of different NN accelerator architectures. Conventional digital architectures have discrete MAC units and memory [Fig. 4(a)] [1], [5], [10], [13], [16], [18], [22]. With the increasing size and computation of NN models, the massive data movements between the MAC units and memory will affect the overall energy efficiency.

CIM is a promising architecture owing to the integration of MAC units and memory. As illustrated in Fig. 4(b), previous CIM-based NN accelerators usually apply analog CIM that implements analog MAC in memory [9], [15], [20], [21], [23]. Input digital-to-analog converters (DACs) and output analog-to-digital converters (ADCs) are designed in the CIM's peripherals. For one NN layer represented by $O = I * W$, inputs are first converted to analog signals and then perform bitwise analog MAC operations with the weight matrix (W). The outputs are converted back to digital in the end. However, due to nonideal issues such as signal margin and process variation, previous analog CIM usually can only support $\leq$8b precision, which limits the application to high-accuracy scenarios. As the requirement for higher accuracy and robustness arises, there is an emerging CIM architecture called digital CIM [6], [11], [17], which embeds bitwise digital MAC to SRAM, as shown in Fig. 4(c). They usually attach a logic gate to each memory cell for bitwise multiplication and place accumulators in the CIM subarray. Digital CIM fuses the benefits of digital architectures and CIM: integrating compute into memory achieves high efficiency; digital in-memory logic avoids analog nonideality and guarantees high accuracy.

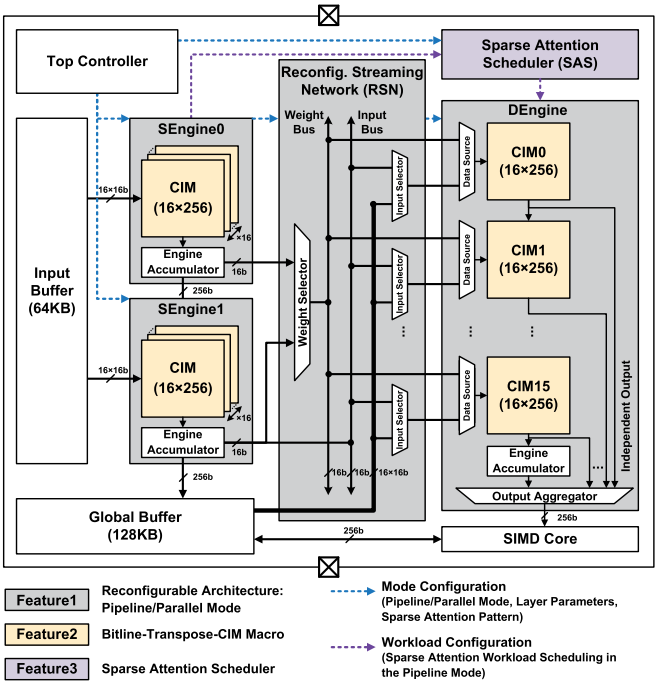


Fig. 5. TranCIM accelerator's overall architecture.

C. Motivation of TranCIM

Considering the challenges in memory access and computation, we will design a full-digital CIM-based transformer accelerator called TranCIM, with pipeline/parallel reconfigurable modes and block sparse attention exploitation. Previous CIM-based accelerators mainly focus on intra-CIM macro design and use a parallel workload mapping style. TranCIM extends more innovations to inter-CIM architecture design and utilizes pipeline/parallel modes for different transformer layers. The sparse attention mechanism is implemented through dynamic CIM workload mapping and also relies on TranCIM's flexible interconnections among CIM macros.

III. TranCIM Accelerator Architecture

The TranCIM (**Tran**sformer **CIM**) accelerator's architecture overview is shown in Fig. 5, with three features highlighted in different colors. It comprises two static engines (SEngine0 and SEngine1), a dynamic engine (DEngine), an RSN, a SAS, a 64 KB input buffer, a 128 KB global buffer, a single instruction multiple data (SIMD) core, and a top controller. Each engine has 16 bitline-transpose-CIM (BLT-CIM) macros (array size: 16 $\times$ 256) and an engine accumulator to add up CIM outputs. Compared with SEngine, DEngine has extra configurations for data source selection and output aggregation, which is specially designed for the pipeline mode with a sparse attention mechanism.

TranCIM has reconfigurable modes for transformer's attention layers and FC layers (**Feature1**). The parallel mode is similar to the work mode of previous CIM-based accelerators [6], [15], [23]. All three CIM engines store the weights

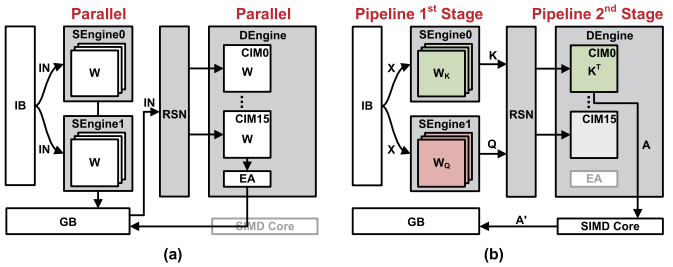


Fig. 6. TranCIM's pipeline/parallel reconfigurable modes (IB: input buffer, GB: global buffer, EA: engine accumulator). (a) Parallel mode (FC layer). (b) Pipeline mode (attention layer).

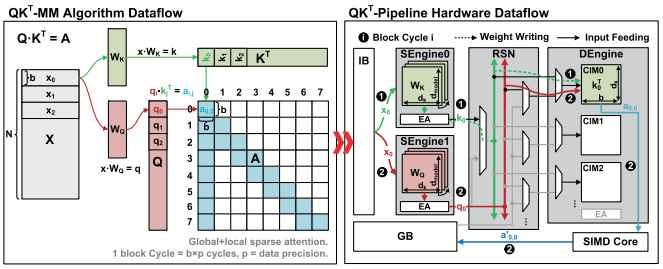


Fig. 7. TranCIM's mapping from QK^T-MM algorithm dataflow to QK^T-pipeline hardware dataflow, with global + local sparse attention.

of an FC layer and work in parallel. The pipeline mode works for the two dynamic MMs in attention layers (QK^T and $A'V$). In the QK^T-pipeline, TranCIM configures SEngine0 and SEngine1 as the 1st stage to compute Matrix K and Q, and DEngine as the 2nd stage to compute Matrix $A = Q \cdot K^T$. The BLT-CIM macro aligns DEngine's input feeding and weight writing directions to avoid additional transpose buffers (**Feature2**). DEngine's CIM macros are configured as independent outputs instead of adding up all outputs. SAS dynamically configures DEngine's CIM workload under the sparse attention mechanism (**Feature3**). The SIMD core performs scaling and softmax on A to generate Matrix $A' = \text{softmax}((A/\sqrt{d_k}))$. The $A'V$-pipeline has a similar dataflow, except that the 2nd stage's input A' is loaded from the global buffer instead of the 1st stage. SEngines are combined to compute Matrix V, while DEngine computes the attention layer's output matrix $\text{Att} = A' \cdot V$. In this article, we will mainly use the QK^T-MM as example to explain the technical features of TranCIM.

IV. Pipeline/Parallel Reconfigurable Modes

A. Parallel Mode

TranCIM's parallel mode works for transformer's FC layers, which is similar to the work mode of previous CIM accelerators [6], [15], [23], as demonstrated in Fig. 6(a). The three CIM engines work independently and store the FC layer weights W. The inputs for SEngine0 and SEngine1 are stored in the input buffer, while the inputs of DEngine are stored in the global buffer. RSN connects DEngine with SEngines and the global buffer, through the buffer ports, input/weight buses, and input/weight selectors. In the parallel mode, RSN disables the connection between engines and activates the input feeding dataflow from the global buffer to DEngine. The engines' outputs are all stored in the global buffer.

This work mode has proven to be effective for static MM in conventional NN layers like transformer's FC layers. However, as mentioned in **Challenge1a**, dynamic MM computation in attention layers will cause redundant OFF-chip memory access for intermediate data. Taking QK^T-MM as an example, K and Q are sequentially generated and written out of the chip. The two matrices are then loaded back to compute A.

B. Pipeline Mode

TranCIM's pipeline mode is specially designed for the transformer's attention layers. The reconfiguration for pipeline

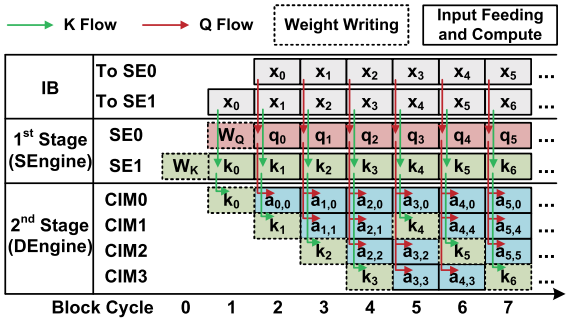


Fig. 8. QK^T-pipeline time chart, with global + local sparse attention.

or parallel modes is realized by changing the dataflow in RSN. In the QK^T-pipeline, as shown in Fig. 6(b), SEngine0 and SEngine1 (1st stage) store weight matrices W_K and W_Q, respectively. By processing the input sequence X from the input buffer, the two SEngines generate Q and K. By dynamically reconfiguring RSN, the two matrices are sent to DEngine (2nd stage) to compute A.

Fig. 7 explains the QK^T-pipeline mode with global + local sparse attention similar to the pattern proposed in [2]. The block sparsity size is b, which means every b token of the input sequence is processed together to compute a $b \times b$ block of the attention matrix A. Here, b is set as 16 to fit the size of a CIM macro. With W_K and W_Q stored in SEngine0 and SEngine1, the b tokens are sent into the engines bit-serially, so it takes $b \times p$ cycles to compute one block (q_i in Q, or k_j in K), where p is the data precision. This period of time ($b \times p$ cycles) is defined as a block cycle. RSN dynamically activates the weight bus and input bus to construct a two-stage pipeline between SEngine and DEngine. DEngine receives q_i, k_j blocks through RSN, and multiplies them on its CIM macros to compute one block of Matrix A ($a_{i,j} = q_i \cdot k_j^T$). Fig. 8 illustrates the first eight block cycles of the example in Fig. 7. Both Q and K are divided into eight blocks (q_0–q_7, k_0–k_7), so Matrix A has $8 \times 8 = 64$ blocks in total. Take $a_{0,0}$ computation for example. In block cycle1, SEngine0 generates k_0, and writes it as the weight in DEngine's CIM0 with RSN activating the SEngine0-Weight Bus-DEngine-CIM0 dataflow. In block cycle2, SEngine1 generates q_0, and RSN activates the SEngine1-Input Bus-DEngine-CIM0 dataflow. Block q_0 is fed as the input for CIM0, and $a_{0,0}$ is computed at the same time.

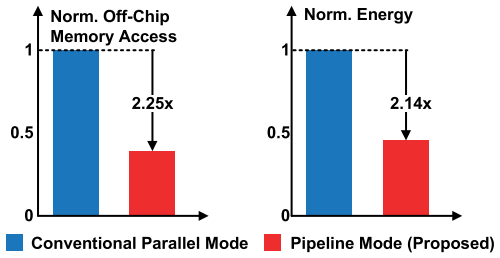


Fig. 9. Evaluation for TranCIM's pipeline mode. Evaluated at 1.0 V, 240 MHz. DDR3 memory is included for OFF-chip memory access evaluation.

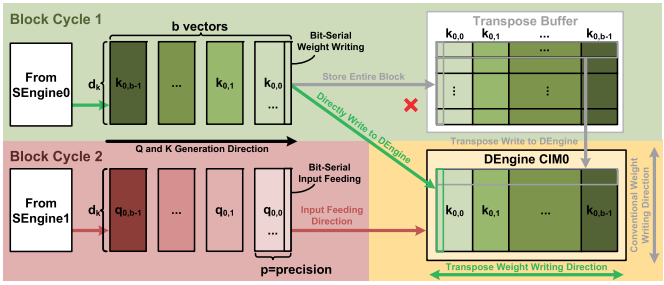


Fig. 10. QK^T-pipeline with two different DEngine architectures: conventional wordline-feeding CIM + transpose buffer architecture (top right), and bitline-transpose CIM architecture (bottom left).

C. Technique Evaluation

Fig. 9 shows the evaluation for TranCIM's pipeline mode. The benchmark is a typical QK^T-MM with parameters of INT16 precision, $N = 1024$, $d_{\text{model}} = 256$, $d_k = 16$, $b = 16$. The baseline is TranCIM's parallel mode (blue bar), which represents the conventional work mode of previous CIM accelerators [6], [15], [23]. Owing to the pipeline mode, the input sequence X is loaded only once to compute Q and K simultaneously. Q and K do not need to write out and can be directly reused on the chip. TranCIM's pipeline mode achieves 2.25× less OFF-chip memory access and 2.14× energy saving.

V. Bitline-Transpose-CIM Macro

A. Wordline-Feeding CIM Versus Bitline-Transpose CIM

Although TranCIM's pipeline mode reduces OFF-chip memory access, **Challenge1b** arises in QK^T computation. Fig. 10 illustrates two sequential block cycles for computing $a_{0,0} = q_0 \cdot k_0^T$. Both k_0 and q_0 blocks have b vectors with vector length of d_k. The SEngines generate the blocks along the b direction. To realize $q_0 \cdot k_0^T$ in DEngine, DEngine's CIM0 should store k_0 in a transposed way: b vectors along the horizontal direction, and d_k elements along the vertical direction. However, conventional CIM macros have a wordline-feeding architecture, which means Block k_0 is written along the vertical (d_k) direction. Thus, as shown on the top-right of Fig. 10, an extra transpose buffer with the same size of CIM0 is needed to store k_0 and then transpose write it to CIM0.

B. BLT-CIM Macro Architecture

To overcome the challenge, we design a bitline-transpose-CIM (BLT-CIM) macro that aligns the input feeding and weight writing directions. Fig. 11(a) shows the BLT-CIM macro architecture which places bitlines horizontally and uses bitlines instead of wordlines to receive inputs for computation. The CIM macro consists of a CIM controller, a 16 × 256 SRAM-CIM array, and 16 macro accumulators. TranCIM's three engines have the same CIM macro architecture, while their difference is the source of the CIM controller's configurations. All the engines receive the top controller's static mode configurations such as pipeline/parallel mode, layer parameters, and sparse attention pattern. In the pipeline mode, DEngine's CIM controller has dynamic CIM workload configurations from the SAS. Unlike previous analog CIM [15], [23], TranCIM's full-digital CIM activates all the rows simultaneously for full array utilization and has no accuracy loss caused by the nonideal issues.

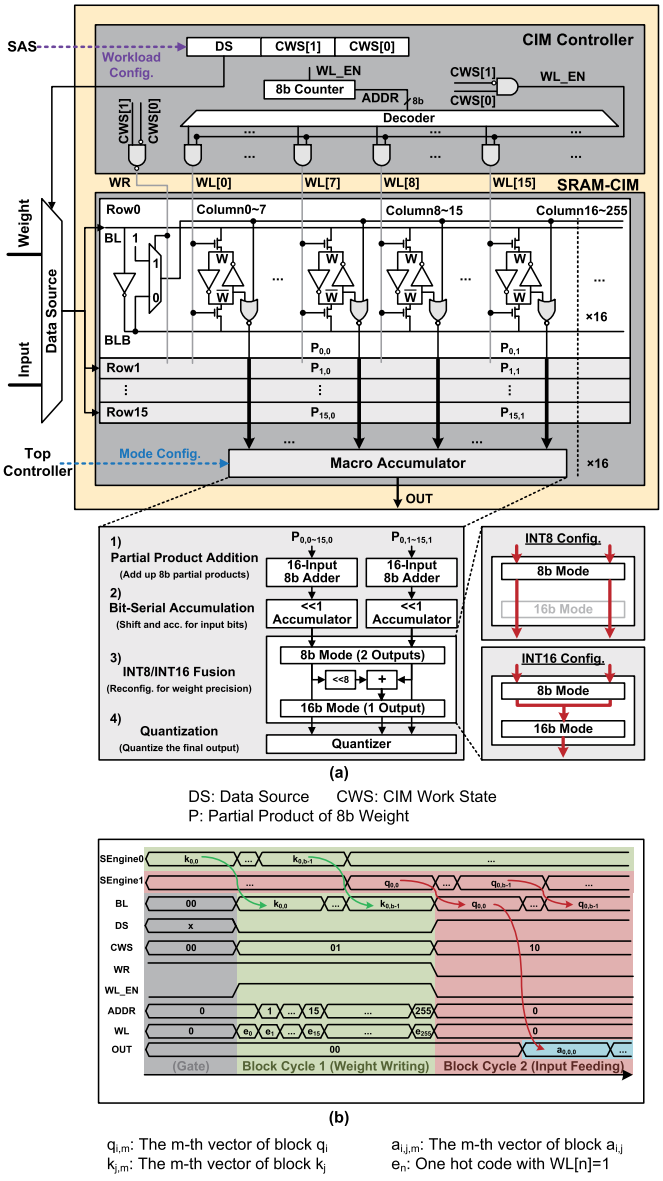


Fig. 11. BLT-CIM macro architecture. (a) DEngine CIM0 view. (b) BLT-CIM macro's operation waveform.

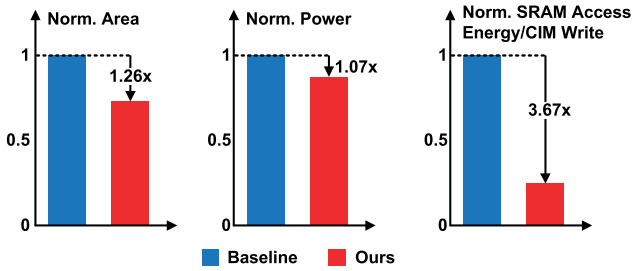


Fig. 12. Evaluation for the BLT-CIM macro at 28 nm, 1.0 V, 240 MHz.

The CIM controller has a 3b configuration word: 1b for the data source (DS) and 2b for the CIM work state (CWS). The DS bit controls the data source multiplexer (left) to select data from the Weight or Input port. The CWS bits indicate the work state as weight writing ("01") or input feeding ("10"). In the weight writing state, the data source is the weight port. The address counter is triggered and the corresponding wordline (WL) is open. In the input feeding state, the data source is the input port, and the in-memory computing logic is activated. Input bits are horizontally sent through the bitlines (BLs) and multiplied with the weights stored in the SRAM cells.

Previous SRAM-CIM is usually analog CIM and requires 8T or 10T SRAM to handle analog computing issues [4], [24]. TranCIM's SRAM array is designed with full-digital in-memory logic. 6T SRAM is applied for the highest weight storage density, and a 4T NOR gate is connected to each SRAM cell for bitwise multiplication. Since SRAM has inherent negation logic in the bitline pair (BL/BLB) and 6T cell ($\overline{W}$ storage), we utilize the feature to realize I AND W with $\overline{I}$ NOR $\overline{W}$, saving two transistors per cell. The bitwise logic occupies only 2.73% area in the total SRAM-CIM array. The SRAM's normal memory access is not affected, because the in-memory logic works only when the word line is not activated (standby status). In our design, each standard 6T SRAM cell is connected with a 4T NOR gate for bitwise multiplication. In the 16 × 256 SRAM-CIM array, every 16 columns have a macro accumulator with four stages.

1) *Partial Product Addition:* Two 16-input 8b adders that perform vertical addition for the 16 rows of partial products.
2) *Bit-Serial Accumulation:* Two accumulators that perform shift-accumulation for the bit-serial inputs.
3) INT8/INT16 fusion that controls how many columns should merge their sums. The INT8 mode directly sends out the two sums of the bit-serial accumulators. The INT16 mode fuses the two sums and generates one output. The reconfiguration overhead is 0.77% area and 0.40% power in total.
4) Quantization for the final outputs.

C. Technique Evaluation

Fig. 12 shows the evaluation for the BLT-CIM macro. The baseline is implemented by replacing DEngine's BLT-CIM macros with 8 KB conventional wordline-feeding CIM macros (similar to TSMC's ISSCC'21-16.4 [6]) and an 8 KB transpose buffer. In the baseline, the outputs of SEngines should be stored in the transpose buffer, and then written to DEngine's CIM macros. In our design, as demonstrated by the example QK^T-MM in Fig. 11(b), k_0 can be directly written into DEngine's CIM0 and then multiplied by q_0 without pipeline stall. TranCIM's BLT-CIM reduces DEngine's area and power by 1.26× and 1.07×, saving 3.67× SRAM access energy consumption for each CIM write. The overhead is only 0.40% area and 0.71% power, mainly due to the CIM controller for the data source and CIM work state configuration.

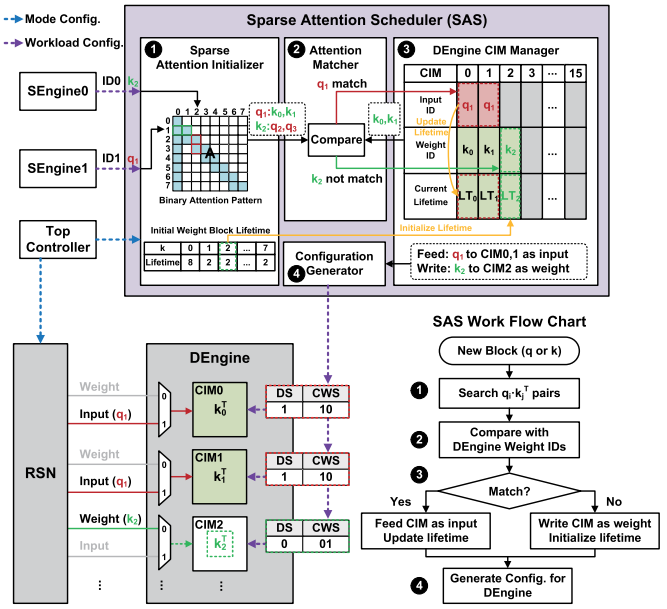


Fig. 13. SAS architecture, work flowchart, and workload configuration for DEngine in the pipeline mode.

VI. Sparse Attention Scheduler

A. Sparse Attention Pattern

In order to overcome the computation bottleneck mentioned in **Challenge2**, we utilize sparse transformer algorithms with block sparse attention [2], [25]. Fig. 7 presents an example QK^T-MM with a typical global + local sparse attention pattern. Both Q and K are divided into eight blocks (q_0–q_7, k_0–k_7). Matrix A has totally $8 \times 8 = 64$ blocks, but only the blue blocks need to compute ($a_{0,0} = q_0 \cdot k_0^T$, $a_{1,0} = q_1 \cdot k_0^T$, etc). The various $q_i \cdot k_j^T$ computation in A demands DEngine make accurate paring for q, k blocks, and perform only necessary computation to realize the algorithm benefits on hardware.

B. SAS Architecture and Work Flow

Since different sparse attention patterns lead to various $q_i \cdot k_j^T$ attention pairs in Matrix A, DEngine's CIM workload always changes between different block cycles in the pipeline mode. Thus, we design a SAS to dynamically configure DEngine's CIM workload. Fig. 13 illustrates how SAS works using QK^T-MM with a global + local sparse attention pattern. During the mode configuration phase of an attention layer, TranCIM's top controller writes the binary attention pattern and weight block

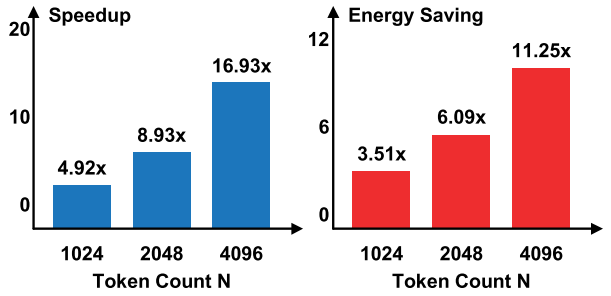
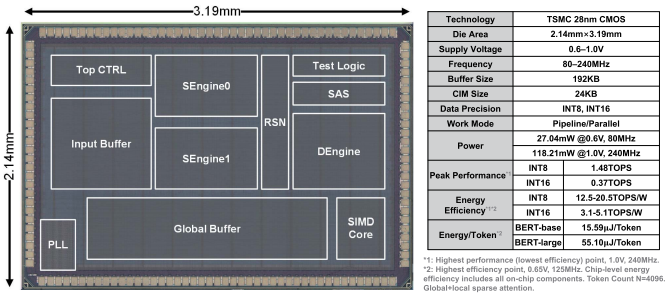


Fig. 14. Evaluation for SAS under different token count at 1.0 V, 240 MHz. OFF-chip memory access is excluded to focus on computation evaluation.



Technology	TSMC 28nm CMOS	
Die Area	2.14mm×3.19mm	
Supply Voltage	0.6–1.0V	
Frequency	80–240MHz	
Buffer Size	192KB	
CIM Size	24KB	
Data Precision	INT8, INT16	
Work Mode	Pipeline/Parallel	
Power	27.04mW @0.6V, 80MHz	
	118.21mW @1.0V, 240MHz	
Peak Performance	INT8	1.48TOPS
	INT16	0.37TOPS
Energy Efficiency*2	INT8	12.5-20.5TOPS/W
	INT16	3.1-5.1TOPS/W
Energy/Token*2	BERT-base	15.59μJ/Token
	BERT-large	55.10μJ/Token

*1: Highest performance (lowest efficiency) point, 1.0V, 240MHz.
*2: Highest efficiency point, 0.65V, 125MHz. Chip-level energy efficiency includes all on-chip components. Token Count N=4096. Global+local sparse attention.

Fig. 15. TranCIM's chip micrograph and specifications.

lifetime (transformer's metadata) into the sparse attention initializer. The binary attention pattern is a binary matrix that indicates which block of the attention matrix is dense ("1," marked in blue) or sparse ("0," marked in white). It's predefined before training with sparse transformer algorithms. The weight block lifetime indicates how many attention blocks are generated by a weight block (e.g., 8 for k_0). The DEngine CIM manager maintains the runtime information of each CIM macro in DEngine: Input ID, Weight ID, and Current Lifetime (LT).

The workflow of SAS has four steps: ❶ In the pipeline mode, once SEngine generates a new block, the sparse attention initializer searches the $q_i \cdot k_j^T$ attention pairs. Fig. 13 demonstrates two sequential block cycles when SEngine0 generates k_2 and SEngine1 generates q_1. According to the binary attention pattern, q_1 attends to k_0 and k_1, while k_2 attends to q_2 and q_3. ❷ The attention matcher compares the attended blocks with the weight blocks in DEngine, to find whether the new block is matched. Since k_0 and k_1 are already stored in DEngine's CIM0 and CIM1, q_1 gets matched. q_2 and q_3 are not stored in DEngine, so k_2 is not matched. ❸ The DEngine CIM manager determines whether to use the new block for input feeding or weight writing, according to the matching result. In the example, the manager sets q_1 as the input for CIM0 and CIM1. Meanwhile, the two CIM macros' weight lifetime (LT_0, LT_1) decreases by one. k_2 has no matched blocks, so it's set as the new weight block for CIM2, and LT_2 is reset with the initial weight block lifetime of k_2. ❹ Based on the manager's updated information, SAS generates configurations for DEngine, controlling each CIM macro's data source and work state. CIM0 and CIM1's 3b configuration word (DS + CWS) is set as "110" to use q_1 for input feeding. CIM2's configuration word is set as "001" to use k_2 for weight writing. In most cases, streaming from SEngine to DEngine is well-pipelined due to the adequate DEngine CIM capacity and strong locality of sparse attention. In case all DEngine macros are busy, the pipeline may stall for a while, while the SEngine accumulator utilizes the local output buffer as a 64-depth queue before sending q, k vectors to DEngine.

C. Technique Evaluation

Fig. 14 shows the evaluation for SAS under different token count. The benchmark is a typical QK^T-MM with parameters of INT16 precision, $N = 1024$, $d_{\text{model}} = 256$, $d_k = 16$, $b = 16$. The token count N is set as 1024, 2048, and 4096. The baseline is TranCIM working in the pipeline mode with full attention, where Matrix A is not blockified and should be computed by $Q \cdot K^T$. Our work uses a global + local sparse attention pattern, similar to the example in Fig. 13. The baseline is TranCIM working in the pipeline mode with full attention. SAS reduces attention computation by assigning only dense and necessary workload to DEngine. The benefits improve with the increase of token count (1024–4096): SAS achieves 4.92×–16.93×speedup and 3.51×–11.25× energy saving over the baseline with full attention.

VII. Experimental Results

This experiment section first describes TranCIM's measurement results from a 28 nm fabricated chip. We then provide the test platform settings for evaluation. We conduct comprehensive studies on the TranCIM accelerator with transformer models like bidirectional encoder representations from transformer (BERT)-base and BERT-large [8], including overall evaluation, layerwise analysis, technique breakdown analysis, and scaling analysis. Finally, we compare TranCIM with state-of-the-art CIM-based accelerators.

A. Chip Micrograph and Summary

Figs. 15 and 16 present the TranCIM chip micrograph, specifications, and voltage-frequency scaling curve. TranCIM is fabricated in 28 nm CMOS technology, with 2.14 × 3.19 mm^2 die area. The chip can work at 0.6–1.0 V supply, 80–240 MHz, with a power consumption of 27.04–118.21 mW. TranCIM is based on full-digital SRAM-CIM with INT8 and INT16 precision support. The peak performance reaches 1.48 TOPS at INT8 and 0.37 TOPS at INT16 under 1.0 V, 240 MHz. The highest energy efficiency is 20.5 TOPS/W at INT8 and 5.1 TOPS/W at INT16 under 0.65 V, 125 MHz. TranCIM's area and power breakdown are depicted in Fig. 17. We build flexible interconnections among CIM macros through RSN, which consumes only 3.2% in area and 0.9% in power. SAS is attached to DEngine for sparse attention support, consuming only 1.9% in area and 1.2% in power.

B. Test Platform Setup

Fig. 18 demonstrates the test platform for the fabricated TranCIM accelerator. The system is composed of a TranCIM test chip, Xilinx VC709 FPGA board, dc power, oscilloscope,

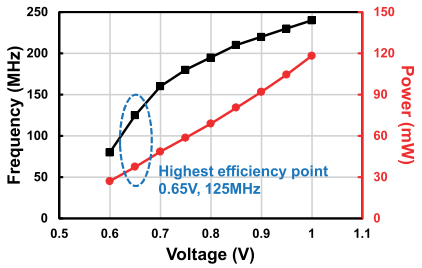


Fig. 16. TranCIM's voltage-frequency scaling curve.

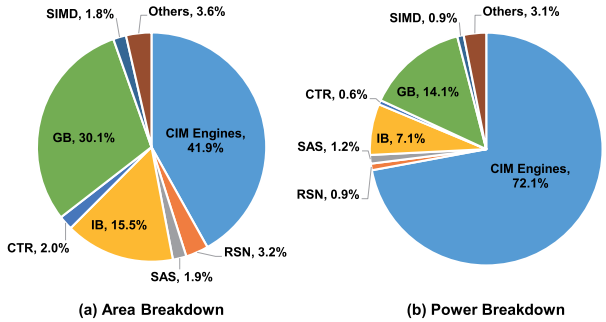


Fig. 17. TranCIM's area and power breakdown. (a) Area breakdown. (b) Power breakdown.

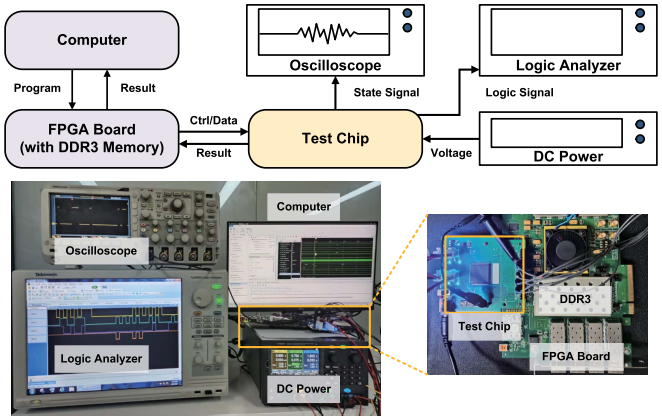


Fig. 18. Test platform for the fabricated TranCIM accelerator.

logic analyzer, and host computer. The test dataset and transformer model are stored in the FPGA board's DDR3 memory, which also serves as the OFF-chip memory for intermediate data. The Xilinx FPGA sends control signals and data to TranCIM via the FPGA mezzanine connector (FMC). The dc power supplies 0.6–1.0 V core voltage for the test chip. TranCIM chip's computing results are collected by the FPGA and then sent to the host computer for further analysis.

C. Evaluation on Transformer Models

1) Overall Evaluation: Table I presents TranCIM's overall evaluation on two typical transformer models, BERT-base and BERT-large [8], with the Natural Questions dataset [12]. To maintain high accuracy, the attention layers use INT16 precision, and the FC layers use INT8 precision. Compared with the F1 score at FP32 precision, our INT8/INT16 quantization has a long answer/short answer (LA/SA) accuracy loss of only $-0.012/-0.008$ on BERT-base and $-0.014/-0.011$ on BERT-large. The results in Row "Execution Time," "Speedup," and "Energy Saving" of Table I are obtained at 1.0 V, 240 MHz for high-performance measurement. We include the DDR3 memory for system-level end-to-end evaluation. The baseline is TranCIM working with the parallel mode and full attention. In comparison, our work runs attention layers in the pipeline mode and utilizes global + local sparse attention. On BERT-base, our work achieves 5.22× speedup and 7.99× energy saving for attention layers, contributing to 2.63× speedup and 2.39× for the entire model. On BERT-large, our work achieves 4.27× speedup and 6.58× energy saving for attention layers, contributing to 2.12× speedup and 1.87× for the entire model. We also measure the TranCIM chip's energy consumption at the highest efficiency point (0.65 V, 125 MHz). The metric here is "Energy/Token," which equals the average energy consumption of processing one input token in the transformer models. TranCIM achieves only 15.59 μJ/Token for on BERT-base and 55.10 μJ/Token on BERT-large. We select NVIDIA Jetson Nano as a GPU baseline, whose peak performance is 0.47TFLOPS (FP16), close to TranCIM's INT16 peak performance of 0.37 TOPS. The GPU has shorter latency due to the higher throughput and DRAM bandwidth. However, TranCIM achieves 83.02× and 77.82× lower energy at the high-efficiency point, owing to the specialized CIM architecture, pipeline mode, and INT8/INT16 precision.

TABLE I
OVERALL EVALUATION ON TRANSFORMER MODELS

Model	BERT-base	BERT-large
Dataset	Natural Questions	
Data Precision	INT16 for attention layers, INT8 for FC layers	
Accuracy (LA/SA)	0.736/0.544	0.765/0.570
Accuracy Loss*1	-0.012/-0.008	-0.014/-0.011
Execution Time*2	1684ms	6646ms
Speedup*2,3	5.22x (attention layers) 2.63x (entire model)	4.27x (attention layers) 2.12x (entire model)
Energy Saving*2,3	7.99x (attention layers) 2.39x (entire model)	6.58x (attention layers) 1.87x (entire model)
Energy/Token*4	15.59μJ/Token	55.10μJ/Token
NVIDIA Jetson Nano Latency and Energy	1208ms 1294.26μJ/Token	3999ms 4287.61μJ/Token

*1: Compared with FP32 accuracy. $N = 4096$. Global+local sparse attention.
*2: Off-chip DDR3 memory is included into latency and energy.
*2: Measured at 1.0V, 240MHz for high-performance evaluation.
*3: The baseline is TranCIM working in the parallel mode with full attention.
*4: Off-chip DDR3 memory is excluded to evaluate chip-level energy consumption.
*4: Measured at 0.65V, 125MHz for high-efficiency evaluation.

2) Layerwise Analysis: Table II presents layerwise analysis on BERT-base and BERT-large inference at 1.0 V, 240 MHz. OFF-chip DDR3 memory is included in latency and energy evaluation. Since the two transformer models are constructed by stacking multiple attention layers and FC layers, we report evaluation results on one stack, and provide breakdown analysis on attention layers (QK^T and $A'V$) and FC layers. For BERT-base, if the token count $N = 1024$, the attention layers with full attention mechanism would take up 57.63% in latency and 45.33% in energy. The sparse attention and TranCIM's

TABLE II
LAYERWISE ANALYSIS ON TRANSFORMER MODELS

Model	Token Count	Layer Type	Memory Access (MB)	Latency (ms)	Energy (mJ)
BERT-base	1024	QK Layer	21.38	8.50	13.35
		A'V Layer	21.75	5.25	13.19
		FC Layer	174.56	21.83	103.47
		One Stack	217.69	35.58	130.01
		Entire Model	2612.25	426.90	1560.12
	4096	QK Layer	78.75	33.46	49.45
		A'V Layer	83.63	20.73	50.76
		FC Layer	683.06	86.13	404.97
		One Stack	845.44	140.32	505.18
		Entire Model	10145.25	1683.89	6062.12
BERT-large	1024	QK Layer	37.50	14.94	23.43
		A'V Layer	37.50	9.12	22.74
		FC Layer	403.00	46.05	238.36
		One Stack	478.00	70.11	284.54
		Entire Model	11472.00	1682.68	6828.98
	4096	QK Layer	138.00	58.84	86.68
		A'V Layer	144.00	36.01	87.45
		FC Layer	1585.00	182.08	937.60
		One Stack	1867.00	276.93	1111.74
		Entire Model	44808.00	6646.35	26681.74

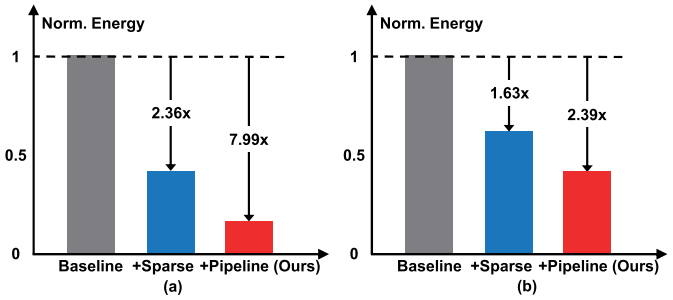


Fig. 19. Technique breakdown analysis on BERT-base ($N = 4096$). (a) Attention layers. (b) Entire model.

pipeline mode significantly reduce the proportion to only 38.64% in latency and 20.41% in energy. If $N = 4096$, full attention will consume 76.66% latency and 66.41% energy in the entire model. Our method can provide more benefits under larger N, decreasing attention layers' proportion to 38.62% in latency and 19.84% in energy. For larger transformer models like BERT-large with $N = 4096$, the conclusion is similar. The attention layers only consume 34.25% in latency and 15.66% in energy under TranCIM's optimization techniques.

3) Technique Breakdown Analysis: Fig. 19 demonstrates a technique breakdown analysis on BERT-base inference with a token count of $N = 4096$, to clearly illustrate how the energy consumption is saved with TranCIM's features. The baseline is implemented on TranCIM by disabling SAS and the pipeline work mode (gray bar). Thus, the baseline can work only in the parallel mode for both attention layers and FC layers in transformer models with full attention, similar to previous CIM-based accelerators [6], [15], [23]. We then optimize the baseline with global + local sparse attention mechanism (blue bar) to reduce the computation complexity of attention layers, achieving energy saving of 2.36× on attention layers. According to Table II, the attention layers' energy proportion is 66.41%, the entire model energy reduction is $1/(66.41\%/2.36 + 33.59\%) = 1.63\times$. Our TranCIM further

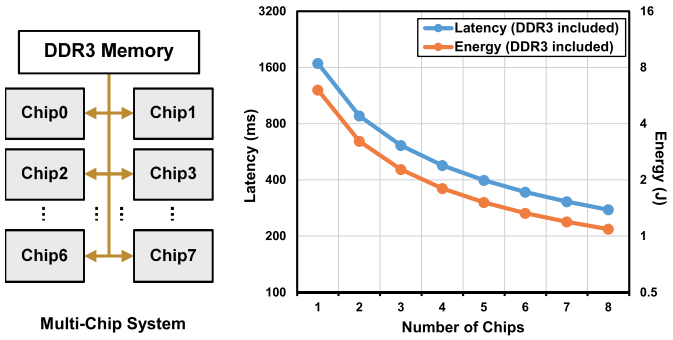


Fig. 20. Latency and energy scaling with increasing TranCIM chip number.

optimizes the baseline with pipeline/parallel reconfigurable modes (red bar) to reduce redundant memory access, obtaining energy saving of 7.99× on attention layers over the baseline, and the entire saving reaches $1/(66.41\%/7.99 + 33.59\%) = 2.39\times$.

4) Latency and Energy Scaling Analysis: Fig. 20 demonstrates our study on latency and energy scalability. We build a simulation system that scales out TranCIM with an increasing number of chips. Each node has the same settings as the measurement results on the single-chip TranCIM, operating at the highest performance point (1.0 V, 240 MHz). The multiple chips are connected to the same DDR3 memory with a bandwidth of 12.8 GB/s. The benchmark is the BERT-base model with token count $N = 4096$ and global + local sparse attention. The workload is partitioned and runs on multiple TranCIM chips in parallel. With the TranCIM chip number increasing from 1 to 8, the latency decreases from 1.68 to 0.28 s, and the system energy consumption decreases from 6.06 to 1.09 J. The curves indicate good latency and energy scaling with increasing TranCIM chips. This is mainly because the memory access for tokens and computation latency has a near-linear relationship with the chip count.

D. Comparison With State-of-the-Art Works

Table III shows a detailed comparison with state-of-the-art CIM-based accelerators. The compared works represent the two mainstream CIM accelerator architectures: analog CIM [15], [23] and digital CIM [6]. The three accelerators all work in the conventional parallel mode without inter-CIM communication, while our TranCIM features digital CIM with pipeline/parallel reconfigurable modes. "Norm. Performance" compares different architectures' performance density under the same technology node, voltage, frequency, and effective INT8 MAC count. The last two rows compare the chip-level energy consumption and end-to-end latency on BERT-base ($N = 4096$). TranCIM consumes only 15.59 μJ/Token for the benchmark, achieving 25.17×, 36.82×, 12.08× energy reduction over [6], [15], [23], respectively. Following is special discussions for each baseline.

ISSCC'21-15.2: [23]'s highest efficiency point is lower than [15], leading to 1.46× lower energy but 5.53× higher latency on BERT-base. TranCIM consumes more power than [23] due to the nearly 2× higher frequency and extra

TABLE III
COMPARISON WITH STATE-OF-THE-ART CIM-BASED ACCELERATORS

	ISSCC'21-15.2 [23]	ISSCC'21-16.3 [15]	ISSCC'21-16.4 [6]	This Work
MAC Implementation	Analog CIM	Analog CIM	Digital CIM	Digital CIM
Data Precision	INT2/4/6/8	INT4/8	INT4/8/12/16	✓ INT8/16
Work Mode	Parallel	Parallel	Parallel	✓ Pipeline/Parallel
Technology	65nm	28nm	22nm	28nm
Supply Voltage (V)	0.62∼1.0	0.7∼0.9	0.72	0.6∼1.0
Frequency (MHz)	25∼100	138.9∼250	500	80∼240
Die Area (mm^2)	12	0.772 (array)	0.202 (macro)	6.69
Power (mW)	18.6∼84.1	Not reported	Not reported	27.04∼118.21
Peak Performance (TOPS)[*1,2]	0.013 (INT8)	0.213 (INT8)	0.917 (INT8), 0.243 (INT16)	1.48 (INT8), 0.37 (INT16)
Norm. Performance (TOPS/mm^2)[*1,3]	0.057 (INT8)	0.063 (INT8)[*4]	0.255 (INT8), 0.071 (INT16)[*4]	0.221 (INT8), 0.055 (INT16)
Energy Efficiency (TOPS/W)[*1,5]	2.75 (INT8/4)	7.29 (INT8), 12.47 (Norm. INT8)[*4]	11.85 (INT8), 3.14 (INT16)[*4]	20.5 (INT8), 5.1 (INT16)
Energy Consumption (μJ/Token) and Latency on BERT-base (s)[*5,6]	392.44 (25.17x)	574.02 (36.82x)[*4]	188.33 (12.08x)[*4]	15.59 (1x)
	357.17 (208.72x)	64.62 (37.76x)[*4]	7.85 (4.59x)[*4]	1.71 (1x)

[*1]: Off-chip DDR3 memory is excluded for chip-level evaluation. One operation (OP) represents one multiplication or one addition.
[*2]: Measured at the highest performance point: 1.0V, 100MHz for [23]. 0.85V, 139MHz for [15]. 0.72V, 500MHz for [6]. 1.0V, 240MHz for TranCIM.
[*3]: All the accelerators are normalized to 28nm, 1.0V, 240MHz, and effective INT8 MAC count for fair comparison.
[*4]: On-chip buffers are included in proportion to the CIM capacity for fair comparison.
[*5]: Measured at the highest efficiency point: 0.65V (digital logic), 1V (CIM), 37.5MHz for [23]. 0.65V, 125MHz for TranCIM. [15] and [6] only work at the same point as [*2].
[*6]: Chip-level energy consumption and end-to-end latency on BERT-base ($N = 4096$). The INT16 results of [15, 23] are projected from their reported INT8 numbers.

logic for transformer support. Future work can lower power consumption by: 1) exploring circuit design at lower operating points; 2) designing more compact CIM layout with shorter data transmission paths; And 3) exploiting more sparsity schemes for a lower toggle rate.

ISSCC'21-16.3: Since nonideal issues cause accumulation errors, analog CIM usually only activates a part of the CIM array simultaneously, while digital CIM can activate the whole array with no accuracy loss. Thus, analog CIM has a larger CIM size when normalized to the same MAC throughput. At iso-precision of INT8, our digital CIM has 3.51× normalized performance and 1.64× energy efficiency over [15].

ISSCC'21-16.4: TranCIM has relatively lower performance density than [6] as we add more components for the proposed techniques. TranCIM has 4.59× speedup on BERT-base because the pipeline mode reduces unnecessary OFF-chip memory access. Beltagy et al. [6] had 12.08× energy mainly due to the large power at 500 MHz. TranCIM has lower frequency for three reasons: 1) Beltagy et al. [6] uses a more advanced technology; 2) TranCIM targets low power prior to high performance; and 3) Beltagy et al. [6] has more compact CIM layout with great engineering optimizations.

VIII. CONCLUSION

The transformer's attention mechanism raises new challenges for CIM-based acceleration. This work designs a full-digital bitline-transpose CIM-based sparse transformer accelerator TranCIM with pipeline/parallel reconfigurable modes. While most previous research focuses on intra-CIM macro design, TranCIM explores a new direction of designing a system with multiple CIM macros and flexible interconnections. The large design space constructed by both intra- and inter-CIM architecture optimizations can be quite meaningful for accelerating large-scale or irregular-structured NNs in the future.

REFERENCES

[1] A. Agrawal et al., "A 7 nm 4-core AI chip with 25.6TFLOPS hybrid FP8 training, 102.4 TOPS INT4 inference and workload-aware throttling," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 144–146.
[2] J. Ainslie et al., "ETC: Encoding long and structured inputs in transformers," in *Proc. EMNLP*, 2020, pp. 268–284.
[3] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020, *arXiv:2004.05150*.
[4] A. Biswas and A. P. Chandrakasan, "Conv-RAM: An energy-efficient SRAM with embedded convolution computation for low-power CNN-based machine learning applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 488–490.
[5] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Jan. 2016, pp. 262–263.
[6] Y.-D. Chih et al., "An 89 TOPS/W and 16.3 TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.
[7] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019, *arXiv:1904.10509*.
[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
[9] J.-M. Hung et al., "An 8-Mb DC-current-free binary-to-8b precision ReRAM nonvolatile computing-in-memory macro using time-space-readout with 1286.4–21.6 TOPS/W for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 182–184.
[10] N. P. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Symp. Comput. Archit.* New York, NY, USA: ACM, 2017, pp. 1–12.
[11] H. Kim, Q. Chen, T. Yoo, T. T.-H. Kim, and B. Kim, "A 1–16b precision reconfigurable digital in-memory computing macro featuring column-MAC architecture and bit-serial computation," in *Proc. IEEE 45th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2019, pp. 345–348.
[12] T. Kwiatkowski et al., "Natural questions: A benchmark for question answering research," *Trans. Assoc. Comput. Linguistics*, vol. 7, no. 29, pp. 453–466, 2019.
[13] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: A 50.6 TOPS/W unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2018, pp. 218–220.
[14] S. Li et al., "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–11.

[15] J.-W. Su et al., "A 28 nm 384kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 250–252.
[16] F. Tu, S. Yin, P. Ouyang, S. Tang, L. Liu, and S. Wei, "Deep convolutional neural network architecture with reconfigurable computation patterns," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 8, pp. 2220–2233, Aug. 2017.
[17] F. Tu et al., "A 28 nm 29.2 2TFLOPS/W BF16 and 36.5 TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 254–256.
[18] F. Tu et al., "Evolver: A deep learning processor with on-device quantization-voltage-frequency tuning," *IEEE J. Solid-State Circuits*, vol. 56, no. 2, pp. 658–673, 2021.
[19] F. Tu et al., "A 28 nm 15.59 μj/token full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 466–468.
[20] C.-X. Xue et al., "A 22 nm 2 MB ReRAM compute-in-memory macro with 121–28 TOPS/W for multibit MAC computing for tiny ai edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 244–246.
[21] C.-X. Xue et al., "A 22 nm 4 Mb 8b-precision ReRAM computing-in-memory macro with 11.91 to 195.7 TOPS/W for tiny ai edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 245–247.
[22] S. Yin et al., "A high energy efficient reconfigurable hybrid neural network processor for deep learning applications," *IEEE J. Solid-State Circuits*, vol. 53, no. 4, pp. 968–982, Apr. 2018.
[23] J. Yue et al., "A 2.75-to-75.9 TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 238–240.
[24] J. Yue et al., "A 65 nm computing-in-memory-based CNN processor with 2.9-to-35.8 TOPS/W system energy efficiency using dynamic-sparsity performance-scaling architecture and energy-efficient inter/intra-macro data reuse," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2020, pp. 234–236.
[25] M. Zaheer et al., "Big bird: Transformers for longer sequences," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 17283–17297.

Fengbin Tu (Member, IEEE) received the B.S. degree from the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China, in 2013, and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing, in 2019.

He is a Post-Doctoral Scholar at the Scalable Energy-efficient Architecture Laboratory (SEAL), Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA, from 2019 to 2022. He has published two books, *Architecture Design and Memory Optimization for Neural Network Accelerators* and *Artificial Intelligence Chip Design*. His research has been published at top conferences and journals on integrated circuits and computer architecture, including International Solid-State Circuits Conference (ISSCC), IEEE JOURNAL OF SOLID-STATE CIRCUITS, Design Automation Conference (DAC), International Symposium on Computer Architecture (ISCA), International Symposium on Microarchitecture (MICRO), and International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). His research interests include artificial intelligence (AI) chip, computer architecture, reconfigurable computing, and computing-in-memory.

Dr. Tu designed the AI chip Thinker and won the 2017 International Symposium on Low Power Electronics and Design (ISLPED) Contest Award. His Ph.D. dissertation was recognized by the Tsinghua Excellent Dissertation Award in 2019.

Zihan Wu received the B.S. degree from the School of Microelectronics, Shandong University, Jinan, China, in 2020. He is currently pursuing the M.S. degree with the School of Integrated Circuits, Tsinghua University, Beijing, China.

His research interests include in-memory computing, computer architecture, and deep learning.

Yiqi Wang received the B.S. degree from the School of Integrated Circuits, Tsinghua University, Beijing, China, in 2021, where he is currently pursuing the Ph.D. degree.

His research interests include deep learning, in-memory computing, and computer architecture.

Ling Liang received the B.S. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2015, and the M.S. degree from the University of Southern California, Los Angeles, CA, USA, in 2017. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA.

His research interests include tensor computing, neural network security, and spiking neural networks.

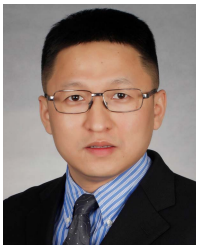
Liu Liu (Member, IEEE) received the B.S. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2013, and the M.S. and Ph.D. degrees from the University of California at Santa Barbara, Santa Barbara, CA, USA, in 2015 and 2022, respectively.

He is currently an Assistant Professor with the Department of Electrical, Computer, and Systems Engineering, Rensselaer Polytechnic Institute (RPI), Troy, NY, USA. His research interests are computer architecture and machine learning, toward high-performance, energy-efficient, and robust machine intelligence.

Yufei Ding received the B.S. degree in physics from the University of Science and Technology of China, Hefei, China, in 2009, the M.S. degree in physics from the College of William and Mary, Williamsburg, VA, USA, in 2011, and the Ph.D. degree in computer science from North Carolina State University, Raleigh, NC, USA, in 2014.

She is currently an Assistant Professor with the Department of Computer Science, University of California at Santa Barbara, Santa Barbara, CA, USA. Her research interests lie in the broad fields of domain-specific language design, architecture and compiler optimization, and hardware acceleration.

Dr. Ding was a recipient of NSF Career Award in 2020, the IEEE Computer Society Technical Community on High Performance Computing (TCHPC) Early Career Researchers Award for Excellence in High-Performance Computing in 2019, the NCSU Computer Science Outstanding Dissertation Award in 2018, the NCSU Computer Science Outstanding Research Award in 2016, and the Distinguished Paper Award at OOPLSA in 2020.



Leibo Liu (Senior Member, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 1999 and the Ph.D. degree from the School of Integrated Circuits, Tsinghua University, in 2004.

He is currently a Professor with the Institute of Microelectronics, Tsinghua University. His research interests include reconfigurable computing, mobile computing, and VLSI Digital Signal Processing (DSP).

Shaojun Wei (Fellow, IEEE) was born in Beijing, China, in 1958. He received the Ph.D. degree from Faculte Polytechnique de Mons, Mons, Belgium, in 1991.

He is currently a Professor with the School of Integrated Circuits, Tsinghua University, Beijing. His main research interests include VLSI SoC design, electronic design automation (EDA) methodology, and communication ASIC design.

Dr. Wei is a Senior Member of the Chinese Institute of Electronics (CIE).

Yuan Xie (Fellow, IEEE) received the B.S. degree in electronic engineering from Tsinghua University, Beijing, China, in 1997, and the M.S. and Ph.D. degrees in electrical engineering from Princeton University, Princeton, NJ, USA, in 1999 and 2002, respectively.

He was an Advisory Engineer with the IBM Microelectronics Division, Burlington, NJ, USA, from 2002 to 2003. He was a Full Professor with Pennsylvania State University, State College, PA, USA, from 2003 to 2014. He was a Visiting Researcher with the Interuniversity Microelectronics Centre (imec), Leuven, Belgium, from 2005 to 2007 and in 2010. He was a Senior Manager and a Principal Researcher with the AMD Research China Laboratory, Beijing, from 2012 to 2013. He is currently a Professor with the Department of Electrical and Computer Engineering, University of California at Santa Barbara, Santa Barbara, CA, USA. His interests include VLSI design, electronic design automation (EDA), computer architecture, and embedded systems.

Dr. Xie is an American Association for the Advancement of Science (AAAS) Fellow and an Association for Computing Machinery (ACM) Fellow. He is an expert in computer architecture who has been inducted to the IEEE/ACM International Symposium on Computer Architecture (ISCA), the IEEE/ACM International Symposium on Microarchitecture (MICRO), and the IEEE International Symposium on High-Performance Computer Architecture (HPCA) Hall of Fame.

Shouyi Yin (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2000, 2002, and 2005, respectively.

He has worked with Imperial College, London, U.K., as a Research Associate. He is currently the Full Professor and the Vice Director of the School of Integrated Circuits, Tsinghua University. His research interests include reconfigurable computing, artificial intelligence (AI) processors, and high-level synthesis. He has published more than 100 journal articles and more than 50 conference papers.

Dr. Yin served as a Technical Program Committee Member in the top VLSI and Electronic Design Automation (EDA) conferences, such as Asian Solid-State Circuits Conference (A-SSCC), International Symposium on Microarchitecture (MICRO), Design Automation Conference (DAC), International Conference on Computer-Aided Design (ICCAD), and Asia and South Pacific Design Automation Conference (ASP-DAC). He is an Associate Editor of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS, *ACM Transactions on Reconfigurable Technology and Systems* (ACM TRETS), and *Integration*, the VLSI journal.